

Variables, Locals & Outputs

Make your Terraform code dynamic and reusable — the first half of the Variables story.

Input variables

Setting values

Validation

Locals & outputs

Lab 03

Today's roadmap



1 - Input variables

Types, optional arguments, and how to declare them well.



2 - Setting values

The precedence order and why several tfvars files exist.



3 - Validation & locals

Guard inputs early; compute reusable local values.



4 - Outputs & Lab 03

Return & protect values; parameterize your project.

01

Input Variables

Parameters for your Terraform code — customize without touching the source.

Variables in Terraform



Inputs

Parameters set from outside —
`var.name`.



Locals

Computed helpers inside the
code — `local.name`.



Outputs

Values returned after apply —
`output`.



Values live **separately** from the plan, so they're easy to read and edit from a single file.

Types & optional arguments

HCL

```
variable "location" {  
  type      = string  
  description = "Azure region"  
  default    = "westeurope"  
}  
  
variable "tags" {  
  type = map(string)  
}
```

- **type** — string · number · bool · list · map · object
- **default** — makes the variable optional
- **description** — documents it
- **validation** — custom rules
- **sensitive** — hides it from output



No default = you **must** supply a value. Terraform has no unassigned variables.

Setting values: precedence

1 · Command line `-var`

Highest precedence — wins over everything

2 · Config files `*.tfvars`

`-var-file`, then `*.auto.tfvars`, then `terraform.tfvars`

3 · Environment variables `TF_VAR_name`

Great for secrets in pipelines

4 · Default in `variables.tf`

Fallback when nothing else is set

5 · Interactive prompt

Last resort — Terraform asks you

tfvars strategy



*.auto.tfvars

Shared defaults that should **always** apply — loaded automatically.



*.tfvars

Environment-specific values (prod / staging / dev) — passed explicitly with `-var-file`.

BASH

```
terraform plan -var-file="prod.tfvars"
export TF_VAR_location="westeurope" # env var
```

Input validation

Define custom conditions inputs must meet — catching bad data at **plan** time, not after resources break.

- **condition** — must evaluate to true
- **error_message** — shown when it fails

HCL

```
variable "env" {  
  type = string  
  validation {  
    condition      = contains(  
      ["dev","prod"], var.env)  
    error_message = "env must be dev or prod."  
  }  
}
```

02

Locals, Outputs & Interpolation

Compute once, reuse everywhere — and return what matters.

Local values

- Defined, assigned and used in the same code
- Accessed as `local.name`
- Set by **you**, not by the user (unlike inputs)
- Change a value once, in one central place

```
HCL
locals {
  prefix = "tbernardo"
  common_tags = {
    owner      = "tiago"
    managed_by = "terraform"
  }
}

# usage
name = "${local.prefix}-app-rg"
```

Outputs & sensitive values

- Expose values after apply, or feed other resources
- Printed automatically on the standard output
- Mark `sensitive = true` to redact secrets



`terraform output name` reveals a sensitive value on demand; `terraform show` reads anything in state.

HCL

```
output "repo_clone_url" {  
  value      = github_repository.course.http_clone_url  
  sensitive = true  
}
```

Direct access vs. interpolation

Direct

Use the value as-is.

```
HCL
location = var.location
```

Interpolation \${...}

Embed inside a string; combine, do math, call functions.

```
HCL
name = "${var.prefix}-rg"
```

💡 Inside `${ }` you can reference variables, run functions, and use conditionals — the gateway to Session 04.

 LIVE DEMO

Variables in action

Walk through `variables.tf`, locals, tfvars files and outputs — plan with `-var-file` and read an output value.

Demo · 03 **Provider** · GitHub

(x) LAB 03 · 30-40 MIN

Parameterize your project

Add variables, locals and outputs to your repo project; use tfvars files to drive different values and expose a clone URL.

You'll practise: variable types · tfvars · locals · sensitive outputs · `terraform output`

Key takeaways

✓ Inputs make code reusable

Typed, documented, optionally validated.

✓ Precedence matters

CLI > tfvars > env > default > prompt.

✓ Locals & outputs

Compute once; return & protect what matters.

✓ Next up

Session 04 — conditionals & iterations.



Questions?

Up next — [Session 04: Conditionals & Iterations.](#)